

Lista de Exercícios 2 (2026/1)

Este trabalho consiste em resolver a lista de exercícios das páginas a seguir, em C.

Para a entrega devem ser seguidas as seguintes regras:

- criar um arquivo compactado no formato ZIP com o nome do aluno no formato *camelHump* (por exemplo, para João Pedro da Silva, usar `JoaoPedroDaSilva.zip`), **SEM SUBDIRETÓRIOS** e **APENAS COM OS ARQUIVOS DE CÓDIGO-FONTE** (NÃO envie quaisquer outros arquivos, como, por exemplo, arquivos `.class`, `.txt`, `README.txt`, `.o` ou executáveis);
- o código-fonte deve ser **ADEQUADAMENTE INDENTADO**;
- o arquivo compactado deve conter programas em C para resolver cada um dos exercícios, salvando o código-fonte em um arquivo com o nome `Exercicio` seguido do número do exercício com **TRÊS dígitos** (por exemplo, `Exercicio001.c`, `Exercicio002.c`, ..., `Exercicio050.c`, ..., `Exercicio101.c`, ...);
- **ATENÇÃO:** os exercícios NÃO seguem necessariamente uma sequência contínua, então tome cuidado de **USAR O NÚMERO CORRETO DO EXERCÍCIO NO RESPECTIVO ARQUIVO DE CÓDIGO-FONTE**;
- **NÃO USAR ACENTOS NO NOME DE ARQUIVOS E DE FUNÇÕES** ;
- no início de cada arquivo em C, incluir um comentário informando o nome do arquivo, o nome do autor, a finalidade do programa e a versão (ou data) de criação (ou atualização);
- quando houver dados a serem lidos, **LER OS DADOS SEMPRE NA MESMA ORDEM EM QUE ELES SÃO CITADOS NO ENUNCIADO**, escolhendo os tipos numéricos adequadamente (se houver dúvida entre usar um tipo inteiro ou um tipo real, use os exemplos de entradas e saídas que aparecem após cada exercício);
- **ESCREVER OS RESULTADOS SEMPRE NA MESMA ORDEM EM QUE ELES SÃO CITADOS NO ENUNCIADO**, escolhendo os tipos numéricos adequadamente (**NÚMEROS REAIS DEVEM SER IMPRESSOS SEMPRE COM 4 CASAS DECIMAIS**, a não ser que seja explicitamente indicado de outra forma);
- na versão final, tomar o cuidado de **NÃO IMPRIMIR NADA DIFERENTE DA SAÍDA ESPERADA** (não devem aparecer, por exemplo, mensagens pedindo que o usuário forneça ou digite determinado valor no terminal);
- a entrega deverá ser feita no dia e horário informado pelo professor em sala de aula e/ou definida na opção de entrega da plataforma moodle da PUCRS.

2. Escreva uma função em C que receba um tempo em total de segundos desde a meia-noite (um valor maior ou igual a 0 e menor ou igual a 86399), retornando **por referência** o número de horas, de minutos e de segundos correspondentes.

Esta função deve ser inserida, e deve funcionar corretamente, no seguinte programa:

```
#include <stdio.h>

void hms(unsigned seg, unsigned *h, unsigned *m, unsigned *s);

int main() {
    unsigned seg, h, m, s;
    scanf("%u", &seg);
    hms(seg, &h, &m, &s);
    printf("%u_%u_%u\n", h, m, s);
    return 0;
}
```

Observe que o código acima apresenta o protótipo da função.

Exemplo(s):

Teste	Entrada	Saída
1	0	0 0 0
2	3661	1 1 1
3	7384	2 3 4
4	18243	5 4 3
5	73840	20 30 40
6	80421	22 20 21
7	54977	15 16 17
8	36001	10 0 1
9	15000	4 10 0
10	86399	23 59 59

4. Escreva uma função em C para um caixa de banco, que recebe um valor inteiro r (maior ou igual a zero e menor ou igual a 199) correspondente a um troco em moedas que deve ser retornado ao cliente. As moedas estão disponíveis nos valores R\$1,00 (100 centavos), R\$0,50 (50 centavos), R\$0,25 (25 centavos), R\$0,10 (10 centavos), R\$0,05 (5 centavos) e R\$0,01 (1 centavo). Determine o número total de cada uma dessas moedas de forma que o número de moedas seja o menor possível. A função deve ser escrita de forma que o as quantidades de moedas sejam retornadas **por referência**.

Esta função deve ser inserida, e deve funcionar corretamente, no seguinte programa:

```
#include <stdio.h>

void quantidadeDeMoedas(unsigned r, unsigned *m100, unsigned *m50, unsigned *m25, unsigned *m10, unsigned *m5, unsigned *m1);

int main() {
    unsigned r, m100, m50, m25, m10, m5, m1;
    scanf("%u", &r);
    quantidadeDeMoedas(r, &m100, &m50, &m25, &m10, &m5, &m1);
    printf("%u_%u_%u_%u_%u_%u\n", m100, m50, m25, m10, m5, m1);
    return 0;
}
```

Observe que o código acima apresenta o protótipo da função.

Exemplo(s):

Teste	Entrada	Saída
1	166	1 1 0 1 1 1
2	165	1 1 0 1 1 0
3	161	1 1 0 1 0 1
4	156	1 1 0 0 1 1
5	116	1 0 0 1 1 1
6	66	0 1 0 1 1 1
7	29	0 0 1 0 0 4
8	54	0 1 0 0 0 4
9	47	0 0 1 2 0 2
10	199	1 1 1 2 0 4

5. Escreva uma função em C para calcular as raízes de uma equação do segundo grau, usando a fórmula de Bhaskara. Lembre-se que para uma equação do segundo grau como:

$$a.x^2 + b.x + c = 0$$

A fórmula de Bhaskara define que as raízes desta equação serão dadas por:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Esta função deve ser inserida, e deve funcionar corretamente, no seguinte programa:

```
#include <stdio.h>
#include <math.h>

int bhaskara(double a, double b, double c, double *r1, double *r2);

int main() {
    double ca, cb, cc, r1, r2;
    int nraizes;
    scanf("%lf%lf%lf", &ca, &cb, &cc);
    nraizes = bhaskara(ca,cb,cc,&r1,&r2);
    if (nraizes == 0)
        printf("NENHUMA raiz real\n");
    else if (nraizes == 1)
        printf("UMA raiz real: %.4lf\n", r1);
    else {
        if ( r1 > r2 ) { double aux = r1; r1 = r2; r2 = aux; }
        printf("DUAS raizes reais: %.4lf %.4lf\n", r1, r2);
    }
    return 0;
}
```

Observe que o código acima apresenta o protótipo da função.

Exemplo(s):

Teste	Entrada	Saída
1	1 -7 10	DUAS raizes reais: 2.0000 5.0000
2	1.0 2.0 -8.0	DUAS raizes reais: -4.0000 2.0000
3	2 -5 2	DUAS raizes reais: 0.5000 2.0000
4	1.0 -1.0 -6.0	DUAS raizes reais: -2.0000 3.0000
5	1 -6 9	UMA raiz real: 3.0000
6	1.0 10.0 25.0	UMA raiz real: -5.0000
7	4 -4 1	UMA raiz real: 0.5000
8	1.0 2.0 1.0	UMA raiz real: -1.0000
9	1 1 1	NENHUMA raiz real
10	1.0 -2.0 5.0	NENHUMA raiz real
11	3 -2 1	NENHUMA raiz real
12	1.0 0.0 4.0	NENHUMA raiz real